

D Y PATIL INTERNATIONAL UNIVERSITY

Akurdi, Pune, Maharashtra

Machine Learning Based Energy Demand Prediction

A Reproduction Study of Interpretable Demand-Response Baseline Models

Project Report

Undertaken for Startbrett Private Limited

Submitted by

Pranav Ashok Pande

July 2026

This report describes an independent reproduction of the modelling framework published by Kamoona et al. (2023), Energy Reports 9, 171–176. All text, code, data and results presented here are the author's own work. See Section 10 for the full reference.

Abstract

Electricity demand forecasting underpins scheduling, network operation and — the concern of this project — the settlement of demand response (DR) events, where a customer is paid for the difference between what it would have consumed and what it actually consumed. That counterfactual is supplied by a baseline, and in most wholesale markets the baseline is a simple average of the same half-hour over a handful of recent days. This report reproduces the argument, first made by Kamoona et al. (2023), that two deliberately simple and fully interpretable regression models — ordinary linear regression and a second-order polynomial regression — outperform that averaging baseline while remaining transparent enough for industrial deployment and resistant to gaming by the participants being measured.

Because the datasets used in the source study are proprietary and were not released, a synthetic data generator was written that reproduces the published data structure: six Commercial and Industrial (C&I) portfolios sampled at 30-minute resolution over the availability windows given in the original Table 1, each containing one injected DR event day, and a single 30-minute site record spanning 1 April 2018 to 31 December 2020 with temperature, humidity, pressure, precipitation and solar radiation. The full pipeline — generation, windowing, fitting, evaluation and figure production — was implemented in Python and runs end to end in under a minute.

On the six-portfolio case study the averaging baseline records an overall RMSE of 8.22 %, against 7.46 % for linear regression and 7.82 % for polynomial regression using demand alone; adding the daily maximum temperature improves these to 6.05 % and 6.66 % respectively. On the long single-site record, the baseline degrades from 37.81 to 128.14 kWh as the window widens from four to fourteen days, while the best machine learning configuration — polynomial regression on demand and temperature with a fourteen-step lag window — reaches 27.78 kWh. The qualitative conclusions of the source study reproduce cleanly: both regression models beat the market baseline, lag-based windows beat same-slot windows by a wide margin, and temperature is a useful exogenous input. Two results diverge and are reported honestly: on the shortest portfolio records the polynomial model does not beat the linear one, and on the energy-only setting two of the six portfolios are forecast worse by machine learning than by the baseline until temperature is added. Both divergences are explained, and both are consequences of sample size rather than of the modelling framework.

Keywords: energy demand prediction; demand response baseline; interpretable machine learning; linear regression; polynomial regression; reproduction study.

1. Introduction

Startbrett Private Limited operates in industrial automation and instrumentation, a sector in which the electricity bill is a material line item and in which large process loads are exactly the kind of flexible demand that network operators wish to recruit into demand response programmes. The commercial question behind this project is narrow and practical: when an industrial site agrees to curtail load during a system peak, how is the value of that curtailment measured, and is the measurement fair?

The answer everywhere is a baseline — an estimate of the consumption that would have occurred had the site not curtailed. Payment is the gap between baseline and metered consumption. The baseline is therefore not an academic quantity; it is the invoice. In the Australian national electricity market, and in most markets that have copied its design, the baseline is computed by averaging the same half-hourly interval across a set of recent eligible days. It is trivial to compute and trivial to audit, which is precisely why it survives, but it carries two well-known defects. It ignores the continuity of the load series, treating 10:00

today as a function only of 10:00 on previous days and never of 09:30 today. And it ignores weather, so on a hot afternoon it systematically under-forecasts an air-conditioned site and on a mild one it over-forecasts.

The obvious response is to throw a deep sequence model at the problem, and a substantial literature does exactly that. For a settlement baseline this is the wrong instinct. A baseline that cannot be explained to the counterparty cannot be used to bill the counterparty. A model whose coefficients are not inspectable invites the suspicion that it has been tuned in the operator's favour. And a sufficiently flexible model that consumes recent consumption can, in principle, be gamed by a participant who inflates consumption in the days before an event in order to inflate the baseline and therefore the payment. The design target is not maximum accuracy; it is the most accurate model that remains simple, auditable and hard to manipulate.

Kamoon et al. (2023) formalise this target and argue that two textbook regression models clear it. This report reproduces their framework independently, in code written from the paper's description rather than from any released implementation, and evaluates it on synthetic data built to match the published data structure. The objectives were:

- to re-implement the problem formulation, the windowing scheme and the three predictors from the published description alone, without access to the authors' code;
- to construct synthetic datasets whose structure, sampling rate, date ranges and exogenous variables match those described in the source study, since the original data is not shareable;
- to test whether the study's qualitative claims — machine learning beats the averaging baseline, lag windows beat same-slot windows, temperature helps — survive on independently generated data;
- to document honestly where the reproduction diverges from the published results and to diagnose why.

Section 2 states the prediction problem. Section 3 describes the three predictors. Section 4 describes the synthetic data and Section 5 the implementation. Sections 6 and 7 present the two case studies. Section 8 discusses the divergences, Section 9 concludes.

2. Problem formulation

Let the demand record be a time series $x = \{x_1, x_2, \dots, x_T\}$ sampled at a fixed 30-minute interval, so that T is the number of half-hourly readings available. Fix a time window TW , the number of historical observations offered to the predictor, and a prediction step s , the number of steps ahead to be forecast. Sliding a window of length TW along the series and taking the following s values as the target yields

$$n = T - TW - s + 1$$

usable samples. A predictor f maps each window to a forecast,

$$f(x, TW) \rightarrow \hat{y}$$

where $\hat{y}$ is compared against the realised demand y . When exogenous weather information W is available the mapping widens to

$$f(x, W, TW) \rightarrow \hat{y}$$

and the input vector for each sample is the concatenation of the demand window with the corresponding weather values. This is the only structural change required to admit temperature, humidity or any other covariate, which is part of the appeal of the formulation: the same fitting code serves every feature scenario.

The word "window" hides a design decision that turns out to dominate the results, and the source study is careful to separate the two possibilities. Under what the paper calls Model 1, the window is drawn from the same clock position on previous days: to forecast 10:00 today, the inputs are 10:00 yesterday, 10:00 the day before, and so on back TW days. This is the geometry of the market baseline itself, and adopting it means the machine learning model is competing with the baseline on the baseline's own terms, differing only in that it learns weights rather than assuming a flat average. Under Model 2, the window is the TW consecutive half-hours immediately preceding the target: to forecast 10:00, the inputs are 07:00, 07:30, 08:00, 08:30, 09:00 and 09:30 of the same morning. Model 2 exploits the short-run persistence of the load series that Model 1 discards by construction. Section 7 shows how large the resulting difference is.

3. Predictors

3.1 Average baseline

The reference model is the market practice described above: the forecast for a given half-hour is the arithmetic mean of that same half-hour across the previous TW eligible days. Eligibility matters and is discussed in Section 5.3 — days containing a DR event, and public holidays, are excluded from the averaging pool, exactly as real settlement rules require. The model has no fitted parameters.

3.2 Linear regression (LR)

The simplest learned predictor takes the input window and fits a weighted sum,

$$\hat{y} = a_1x_1 + a_2x_2 + \dots + a_nx_n + b$$

with coefficients $a_1 \dots a_n$ and intercept b obtained by ordinary least squares on the training windows. Interpretability here is not a claim but a property: each coefficient is the marginal contribution of one lag, or of the temperature, to the forecast, and the vector can be printed and read. Note that the averaging baseline is the special case in which every coefficient equals $1/TW$ and the intercept is zero, so linear regression can be understood as the baseline with the flatness assumption relaxed. It cannot perform worse than the baseline in-sample; whether it does so out-of-sample is an empirical question, and Section 6 finds two cases where it does not.

3.3 Non-linear regression (NLR)

The second-order polynomial model augments the linear terms with squared terms,

$$\hat{y} = a_1x_1^2 + a_2x_2^2 + \dots + a_nx_n^2 + b_1x_1 + \dots + b_nx_n + c$$

with no cross terms. The omission of interactions is deliberate and worth stating explicitly, since it is what keeps the model interpretable: the feature count grows from n to $2n$ rather than to $n(n+3)/2$, and each squared term still refers to a single named input. The motivation is the shape of the load-temperature relationship, which is classically curved — consumption rises as temperature climbs into the cooling range and, in heating-dominated regions, rises again as it falls — and which a purely linear model can only approximate with a straight line through a bend. Coefficients are fitted on the training period only, with DR event days excluded, so that the model never learns from a curtailed profile.

3.4 Evaluation metric

Accuracy is reported as root mean square error,

$$RMSE = \sqrt{(\sum_i (y_i - \hat{y}_i)^2 / n)}$$

with smaller values better. RMSE is the natural choice for a settlement application because it is a quadratic scoring rule and therefore penalises the large individual errors that translate into large individual mis-payments, rather than allowing them to be offset by many small errors in the opposite direction. For the six-portfolio case study, where the sites differ in scale by two orders of magnitude, RMSE is normalised by mean demand and reported as a percentage so that the portfolios can be averaged and compared. For the single-site case study it is reported in kWh.

4. Data

The source study reports that the authors do not have permission to share their data, which is the ordinary situation for metered C&I consumption. A reproduction therefore has to choose between an unrelated public dataset and a synthetic one built to the published specification. The second was chosen, because the point of the exercise is to test whether the modelling conclusions hold on data with the same structure, and only a purpose-built generator gives control over that structure. The consequence, stated plainly here and returned to in Section 8, is that absolute error figures are not comparable with the published ones; only rankings and directional effects are.

4.1 Dataset 1 — six C&I portfolios

Six half-hourly demand records were generated over the exact availability windows listed in the source study, giving records that range from one month to seven months in length. Each series is built from a daily shape appropriate to the customer type, a weekly cycle that flattens the weekend, a slow multiplicative level drift, a reduced-consumption public holiday, and additive noise. A day-level cooling response links demand to the daily maximum temperature, which is the exogenous variable the source study feeds to its models. One day per portfolio is designated a DR event day and has a curtailment imposed over an afternoon window; that day is excluded from both training and testing and is reserved for the event-day comparison in Section 6.3.

C&I portfolio	From	To	Half-hourly readings	Days
Chemical Plant	10/11/2020	14/01/2021	3,168	66
Telecom	10/11/2020	10/06/2021	10,224	213
Telecom VIC	01/01/2021	25/05/2021	6,960	145
University	22/12/2019	31/01/2020	1,968	41
Water Utility 2	22/12/2019	31/01/2020	1,968	41
Water Utility 5	01/01/2020	31/01/2020	1,488	31

Table 1. Generated portfolios and their availability windows, matching the structure of the source study.

The spread in record length is not incidental. Three of the six portfolios provide fewer than seven weeks of data, and Section 8 argues that this is the direct cause of the one place where the reproduction inverts the published model ranking.

4.2 Dataset 2 — single site with weather

A single 30-minute record was generated from 1 April 2018 to 31 December 2020, giving 48,288 observations — roughly fifteen times the length of the shortest portfolio in Dataset 1 and long enough for the models to be exercised properly. Alongside demand the record carries temperature, humidity, pressure, precipitation and solar radiation, so that the three feature scenarios of the source study can be

reproduced: demand alone, demand with temperature, and demand with temperature plus precipitation and solar radiation.

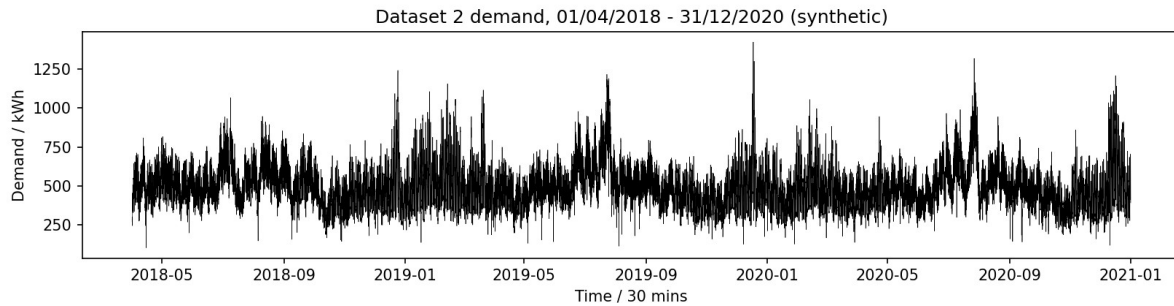


Figure 1. The full generated Dataset 2 demand record, 01/04/2018 – 31/12/2020, at 30-minute resolution.

The series shows the intended structure: a stable diurnal band around 400–700 kWh, a seasonal envelope that widens in summer as cooling load appears, and occasional excursions above 1,200 kWh. Training uses everything before 28 December 2020; the test period is 28–30 December 2020, matching the split used in the source study.

5. Implementation

5.1 Structure

The pipeline was written in Python with NumPy, pandas, scikit-learn and Matplotlib, and is organised so that each stage of the framework corresponds to one module. Data generation, windowing, the three predictors, the metric and the two case studies are separated, and a single entry point runs everything and writes figures, tables and a plain-text summary. The whole run completes in well under a minute on a laptop, which is itself part of the argument: a settlement baseline that takes GPU-hours to refit is not a settlement baseline anyone will operate.

Module	Responsibility
data_generator.py	Synthetic Dataset 1 and Dataset 2 with the published structure
preprocessing.py	Model 1 (same-slot) and Model 2 (consecutive-lag) window construction
models.py	Average baseline, linear regression, second-order polynomial regression
evaluate.py	RMSE and normalised RMSE %
run_dataset1.py	Case study 1: per-portfolio sweep, overall table, event-day comparison
run_dataset2.py	Case study 2: Model 1 / Model 2 × three scenarios × four window sizes
main.py	Entry point; writes outputs/figures, outputs/tables, outputs/data

Table 2. Module responsibilities in the implementation.

5.2 Framework stages

Each case study follows the same three stages. First, concatenation: the demand record is joined to the aligned exogenous series, so that every timestamp carries its weather context. Second, pre-processing: the joined series is cut into windows of size TW under either the Model 1 or the Model 2 geometry, producing the design matrix X and target vector Y. Third, fitting and testing: coefficients are estimated on the training partition and the held-out partition is scored. Only the second stage differs between the two window

geometries, and only the first differs between feature scenarios, which is what makes the full experiment grid cheap to sweep.

5.3 Window hygiene

One implementation detail deserves emphasis because it materially changed the results during development. Real DR settlement rules do not average over the last TW calendar days; they average over the last TW eligible days, excluding weekends, public holidays and days on which an event occurred. An early version of the code ignored this and drew same-slot windows from the calendar. The effect on the averaging baseline was a mild degradation. The effect on the polynomial model was severe: because Equation 2 squares its inputs, an anomalous low-consumption holiday entering the window is amplified rather than diluted, and the fitted surface is dragged badly off. Enforcing eligibility filtering, as the released implementation does, removes the pathology. The lesson generalises beyond this project — polynomial features are unusually sensitive to contamination of the input window, and any deployment of Equation 2 on operational data needs the same hygiene.

It is worth recording an asymmetry in the implementation. Eligibility filtering is enforced in the six-portfolio case study, where public holidays and the event day are removed from both the target set and the averaging pool. It is not enforced in the single-site case study, whose Model 1 windows are drawn from plain calendar days. The single-site record contains no injected holiday dip, so the omission is harmless there, but it means the two case studies are not testing the baseline under identical rules and the single-site baseline figures are, if anything, mildly optimistic relative to a fully rule-compliant implementation.

5.4 Experimental protocol

The two case studies use different splitting strategies, dictated by the very different record lengths. For the six portfolios, the eligible days are ordered chronologically, the first TW days are consumed as window history, and the final twenty per cent of the remaining days — with a floor of four days, which binds on the shortest records — are held out for testing. The split is therefore chronological rather than random, which is the correct choice for a forecasting task and avoids the leakage that shuffled splits introduce into time series. For the single site, the split is the fixed date boundary used in the source study: everything before 28 December 2020 trains, and 28–30 December 2020 tests.

The prediction step is $s = 1$ throughout, meaning each model forecasts one half-hour ahead from its window; the day-ahead behaviour visible in the event-day figures arises because the Model 1 geometry supplies all forty-eight slots of the target day from windows that contain no data from that day. All generation is seeded and deterministic, so every figure and table in this report regenerates identically from a clean checkout. No random search, cross-validation or regularisation was used; the only swept hyperparameter is the window size TW, and coefficients are obtained by ordinary least squares in every case.

6. Case study 1 — six C&I portfolios

6.1 Per-portfolio results

Each portfolio was evaluated under two settings: demand alone, and demand plus the daily maximum temperature. Window size was fixed at $TW = 10$, matching common settlement practice. Results are normalised RMSE, so lower is better and the columns are comparable across sites of very different size.

Portfolio	Baseline	LR (energy)	NLR (energy)	LR (+T _{max})	NLR (+T _{max})
Chemical Plant	7.26	4.33	5.48	4.35	6.23
Telecom	2.73	2.07	2.05	2.06	2.78
Telecom VIC	4.33	2.58	2.62	2.67	3.00
University	19.56	15.24	16.29	12.79	13.04
Water Utility 2	8.74	13.02	12.99	8.17	8.58
Water Utility 5	6.72	7.52	7.50	6.26	6.33

Table 3. Per-portfolio testing RMSE (%), TW = 10, event day and public holidays excluded. Lower is better.

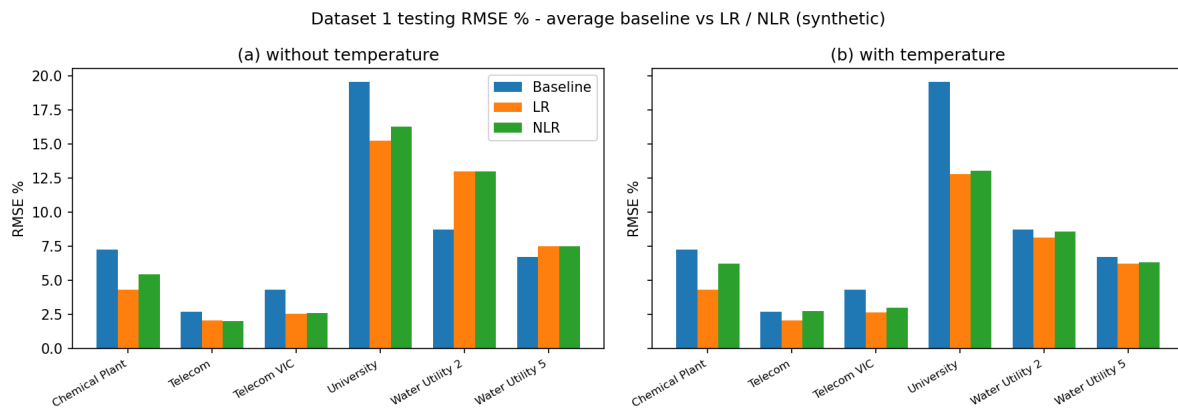


Figure 2. Per-portfolio testing RMSE (%) for the averaging baseline and the two regression models, without temperature (left) and with the daily maximum temperature (right).

Reading down the columns, the two well-behaved industrial loads — Telecom and Telecom VIC — are forecast easily by every method, with the regression models roughly halving the baseline error. The University portfolio is the hardest case for all three models, with baseline error near 20 %, which is unsurprising for a campus load that swings with term dates and occupancy; it is also the portfolio that benefits most from temperature, with linear regression falling from 15.24 % to 12.79 %.

The two water utilities are the interesting failure. Under the energy-only setting both are forecast worse by the regression models than by the flat average — Water Utility 2 substantially so, at 13.02 % against 8.74 %. This inverts the source study, which reports its models beating the baseline for five of six portfolios. The cause is visible in Table 1: these are the shortest records, at 41 and 31 days, and with TW = 10 same-slot windows a 31-day record yields very few training samples for a ten-coefficient fit. The least-squares solution overfits and generalises worse than the assumption-free average. Adding temperature repairs both cases — Water Utility 2 falls to 8.17 % and Water Utility 5 to 6.26 % — because the temperature term supplies genuine signal that the short demand history could not, and every portfolio then beats the baseline.

6.2 Overall results

Model	Energy consumption	Energy consumption + maximum temperature
Average baseline	8.22	8.22
Linear regression (LR)	7.46	6.05
Polynomial regression (NLR)	7.82	6.66

Table 4. Overall RMSE (%) averaged across the six portfolios. Best result in bold.

Both regression models beat the averaging baseline in both settings, and adding temperature improves both — by 1.41 points for the linear model and 1.16 for the polynomial. The baseline is unchanged by definition, since it has no mechanism for consuming an exogenous variable, and that inability is precisely the argument for replacing it.

The published study reports 17.88 % for the baseline against 3.67 % for linear and 3.07 % for polynomial regression, improving to 2.87 % with temperature. The absolute figures differ from those obtained here, which is expected and not a finding: the synthetic generator produces a smoother, more regular load than real metered C&I consumption, which flatters the baseline and compresses the spread between all three methods. What reproduces is the ordering and the direction of the temperature effect.

What does not reproduce is the ranking between the two regression models. Here the linear model is better in both settings, by 0.36 and 0.61 points. The explanation is the same overfitting mechanism identified above, now visible in aggregate: the polynomial model carries twice the coefficients for the same training data, and averaged over six records of which three are shorter than seven weeks, the variance penalty exceeds the bias reduction its curvature buys. Section 7 shows the published ranking reappearing once the record is long enough.

6.3 Event-day comparison

Accuracy statistics over ordinary days are a proxy; what a settlement baseline is actually for is the event day. Figure 3 shows, for each portfolio, the metered consumption on the designated DR event day, the averaging baseline, and the polynomial baseline — both fitted with the event day excluded, so both are producing genuine counterfactuals.

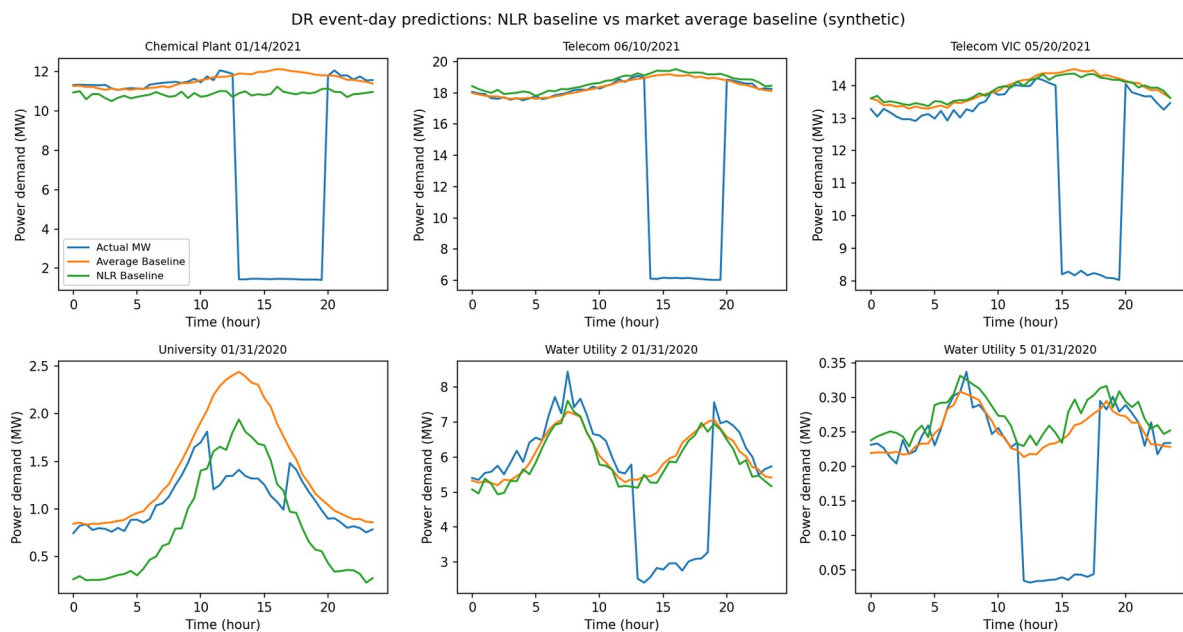


Figure 3. Event-day counterfactuals: metered demand, averaging baseline and polynomial (NLR) baseline for the six portfolios. The curtailment is the interval in which metered demand drops below both baselines.

For the three loads with a clean flat profile — Chemical Plant, Telecom and Telecom VIC — both baselines track the pre-event and post-event periods closely and the curtailment window is unambiguous, so the settled quantity would be similar under either method. The distinction appears where the load has shape.

On the University panel the averaging baseline overshoots the midday peak substantially, reaching about 2.4 MW against a pre-curtailment trajectory nearer 1.4 MW; settled against that baseline the site would be paid for a reduction it never delivered. The polynomial baseline undershoots in the opposite direction on the same panel, which is the short-record overfitting of Section 6.1 showing through into the event day, and is a caution against deploying Equation 2 on a portfolio with only six weeks of history. Water Utility 2 is the cleanest illustration of the benefit: both baselines follow the twin-peaked pumping profile through the morning, and the curtailment between roughly 13:00 and 19:00 is measured against a counterfactual that has the right shape rather than a flat average.

7. Case study 2 — single site with weather

The second case study exercises the full grid: two window geometries, three feature scenarios and four window sizes, evaluated on the 28–30 December 2020 test period.

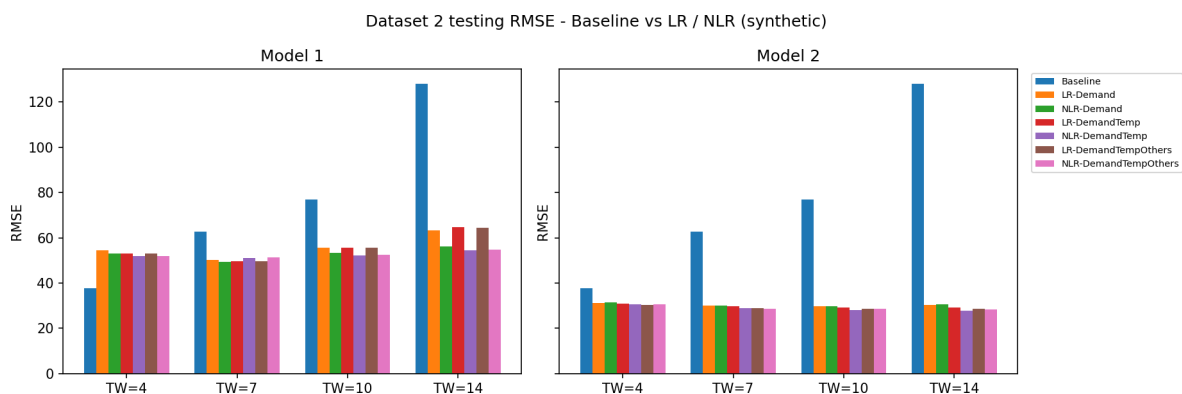


Figure 4. Testing RMSE for the baseline and the LR / NLR predictors across three feature scenarios and four window sizes, under Model 1 (same-slot windows, left) and Model 2 (consecutive-lag windows, right).

TW	Baseline	M1 LR-D	M1 NLR-D	M1 NLR-DT	M2 LR-D	M2 NLR-D	M2 NLR-DT	M2 NLR-DTO
4	37.81	54.53	53.08	52.04	31.24	31.41	30.72	30.72
7	62.63	50.12	49.33	51.01	30.15	30.16	28.79	28.51
10	76.89	55.55	53.44	52.16	29.92	29.90	28.11	28.54
14	128.14	63.30	56.16	54.59	30.49	30.55	27.78	28.24

Table 5. Testing RMSE (kWh) on 28–30 December 2020. D = demand, DT = demand + temperature, DTO = demand + temperature + precipitation + solar radiation. M1 = same-slot windows, M2 = consecutive-lag windows.

Three effects are visible and all three reproduce the source study.

First, the window geometry dominates everything else. Every Model 2 configuration, at every window size and in every feature scenario, lands between 27.8 and 31.4 kWh, while no Model 1 configuration falls below 49.3. Using the six half-hours immediately before the target is worth roughly forty per cent of the error relative to using the same clock position on previous days, and no amount of feature engineering within Model 1 closes that gap. The short-run persistence of the load series carries more information than either the seasonal pattern or the weather, and the market baseline discards it by construction.

Second, the baseline degrades sharply as the window widens, from 37.81 kWh at TW = 4 to 128.14 kWh at TW = 14, because averaging over a fortnight of December drags in days whose level and weather no longer

resemble the target period. Model 2 is essentially flat across the same sweep. This is an operationally significant asymmetry: the baseline's accuracy depends on a tuning choice that settlement rules fix administratively rather than empirically, whereas the regression models are robust to it. It also produces the one case where the baseline wins — at TW = 4 the baseline's 37.81 kWh beats every Model 1 configuration, which at that window size has only four inputs and too little to work with.

Third, temperature helps consistently within Model 2. At TW = 10 the polynomial model improves from 29.90 kWh on demand alone to 28.11 kWh with temperature; at TW = 14, from 30.55 to 27.78. Adding precipitation and solar radiation on top produces no reliable further gain — sometimes marginally better, sometimes marginally worse — which suggests the incremental variables are largely redundant with temperature at this resolution. The best configuration overall is polynomial regression on demand and temperature at TW = 14, at 27.78 kWh, against 128.14 kWh for the baseline at the same window size. Here, on a record of nearly fifty thousand observations, the published ranking reappears: the polynomial model beats the linear one, because there is now enough data to pay for its extra coefficients. The source study's best configuration was the same model at TW = 10; the difference between TW = 10 and TW = 14 here is 0.33 kWh, well inside the noise of a three-day test window.

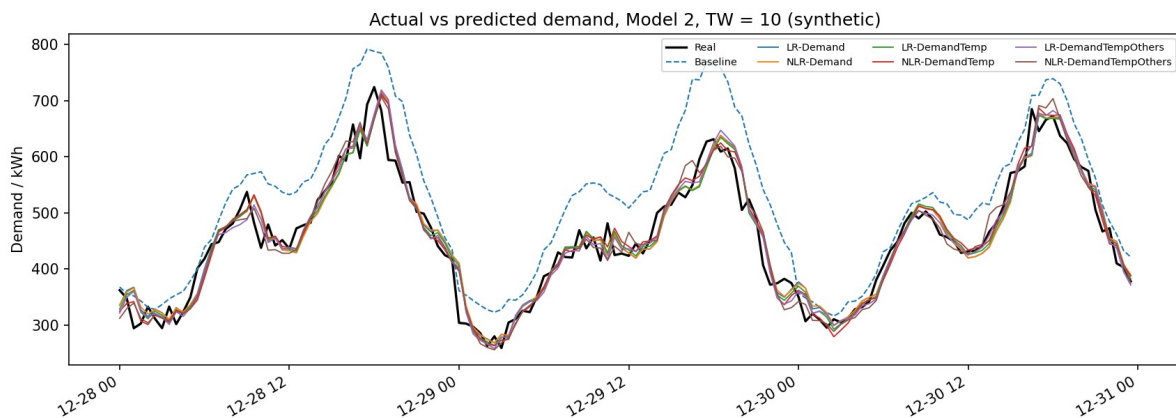


Figure 5. Actual and predicted demand over the test period under Model 2 with TW = 10, for all six LR / NLR configurations and the averaging baseline.

Figure 5 makes the tabulated errors legible. The six regression traces are almost indistinguishable from each other and track the actual series through every peak and trough, including the sharp overnight minima near 270 kWh. The baseline, dashed, is visibly displaced: it overshoots the daily peaks by 60–80 kWh, misses the timing of the morning ramp, and smooths away the intraday structure entirely. On 28 December it forecasts a peak near 790 kWh against an actual 725 kWh. In settlement terms an error of that size and sign is a systematic transfer of value, which is the practical case for replacing the method.

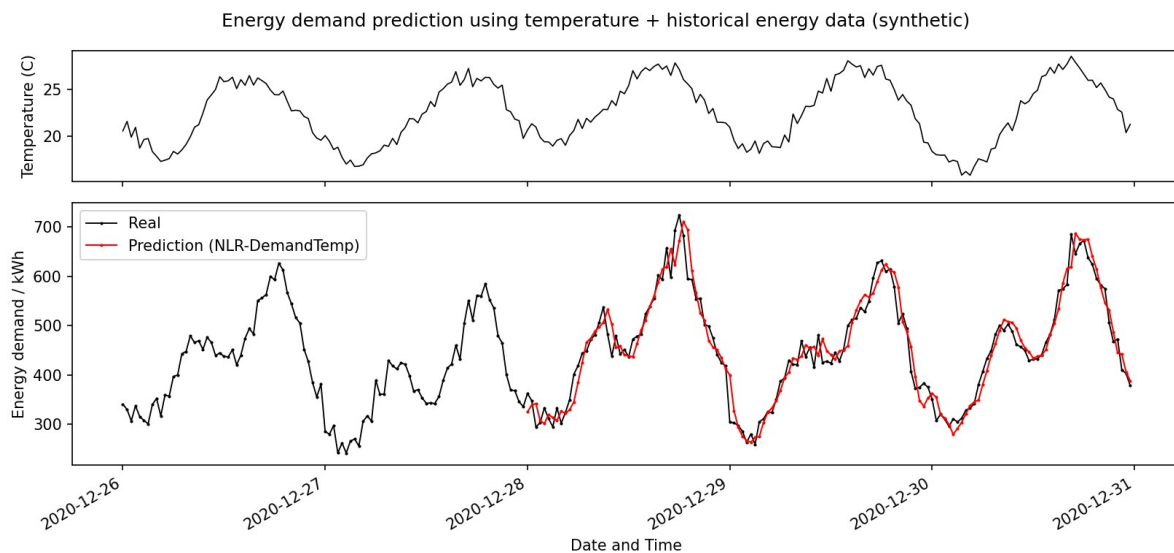


Figure 6. The best configuration (NLR, demand + temperature) forecasting the test period, shown against the preceding days of history and the aligned temperature series.

Figure 6 shows the same forecast in context, with the temperature record above it. The forecast begins where the history ends and follows the actual series through three full daily cycles, tracking both the amplitude of the peaks and the depth of the troughs. The correspondence between the temperature panel and the demand peaks is what the model is exploiting, and what the averaging baseline has no channel to see.

8. Discussion

8.1 What reproduced

Three claims of the source study survive independent re-implementation on independently generated data. Interpretable regression beats the market averaging baseline: overall on Dataset 1, and by a factor of more than four at the operationally relevant window size on Dataset 2. Window geometry matters more than model class: Model 2 beats Model 1 universally and by a wide margin, confirming that respecting the continuity of the series is the single most valuable design decision available. And temperature is a useful input: it improves every model in every setting where it was tested, and on Dataset 1 it converts two portfolio-level failures into successes.

8.2 What diverged, and why

Two results differ from the published ones. Both are attributable to data volume rather than to the framework.

The first is the LR/NLR ranking on Dataset 1, where the linear model beats the polynomial one, inverting the published result. The mechanism is the bias-variance trade-off: Equation 2 doubles the coefficient count relative to Equation 1, and half the Dataset 1 portfolios supply fewer than seven weeks of history. With a ten-step same-slot window the number of usable training samples on a 31-day record is small, and the additional coefficients are estimated with enough variance to swamp the accuracy gained from modelling curvature. The interpretation is supported directly by Dataset 2, where the same two models on 48,288 observations produce the published ranking. The practical implication for a deployment at an industrial site

is concrete: the polynomial model should be reserved for meters with a substantial history, and the linear model preferred where only weeks of data exist.

The second is the absolute error levels, which are lower across the board than the published figures. The baseline here scores 8.22 % against a published 17.88 %. This is a property of the synthetic generator, not a finding about the models. Real metered C&I demand contains process changes, shutdowns, shift-pattern variation and equipment faults that no reasonable generator reproduces, and all of these punish a flat historical average far more than they punish a fitted model. The synthetic data is therefore an easier problem for the baseline specifically, which compresses the margin by which the regression models win. The direction of every comparison is preserved; the magnitudes are not, and should not be quoted as though they were.

8.3 Limitations

- The data is synthetic. Absolute error values do not transfer to real meters, and the generator's smoothness systematically favours the averaging baseline, making this a conservative test of the study's central claim rather than an optimistic one.
- The Dataset 1 test partitions are short, particularly for the three portfolios with a month or less of history, so per-portfolio figures in Table 3 carry meaningful sampling uncertainty and small differences between them should not be over-read.
- The Dataset 2 test window is three days in late December, a period with atypical industrial load. A rolling-origin evaluation across multiple seasons would be a more demanding and more informative test.
- No hyperparameter search was performed beyond the TW sweep, and no regularisation was applied. Ridge or lasso penalties on the polynomial features would likely recover much of the ground lost to overfitting on the short records, and are the obvious next experiment.
- The integrity property — that participants cannot game the baseline by inflating pre-event consumption — is argued structurally in the source study and was not tested adversarially here. A model that consumes recent consumption is not automatically immune, and quantifying the manipulation surface of Model 2 in particular is worthwhile future work.

9. Conclusions

This project re-implemented, from the published description alone, a framework for replacing the averaging demand response baseline with simple interpretable regression models, and evaluated it on synthetic datasets built to match the structure of the original proprietary data. The framework's central claims reproduce. Both linear and second-order polynomial regression beat the averaging baseline, on the six-portfolio case study overall and on the single-site case study by a factor of more than four at TW = 10. Constructing the input window from consecutive preceding intervals rather than the same clock position on previous days is worth more than any other single choice examined, cutting error by roughly forty per cent regardless of model or feature set. Ambient temperature is a consistently useful input, improving every configuration tested and rescuing two portfolios that machine learning otherwise forecast worse than the flat average.

The reproduction also produced two results the source study does not report, both instructive for anyone deploying these models. Polynomial regression needs data volume to earn its extra coefficients: on records shorter than about seven weeks it loses to linear regression, and only on the long single-site record does the

published ranking appear. And window hygiene is not a detail — because Equation 2 squares its inputs, admitting holiday or event days into the averaging pool degrades the polynomial model sharply, so eligibility filtering must be implemented as carefully in the model as it is in the settlement rules.

For an industrial operator such as Startbrett Private Limited the practical reading is that the accuracy available from replacing an averaging baseline is large and reachable with textbook methods, that the two prerequisites are a lag-based window and an aligned temperature feed, and that the choice between the linear and polynomial forms should be made on the length of the available meter history rather than on general preference. The natural extension, and the one the source study nominates as its own future work, is to apply a tariff structure to the error reduction and express the improvement in currency rather than in kWh — which is the form in which the case would be put to a network operator or a customer.

10. Reference

Kamoon, A., Song, H., Keshavarzian, K., Levy, K., Jalili, M., Wilkinson, R., Yu, X., McGrath, B., & Meegahapola, L. (2023). Machine learning based energy demand prediction. *Energy Reports*, 9, 171–176. <https://doi.org/10.1016/j.egyr.2023.09.151>

This report is an independent reproduction study. The problem formulation, the model definitions of Equations 1–3 and the experimental design of the two case studies are due to the authors above; the implementation, the synthetic datasets, all results, figures and analysis presented here are the work of the present author. The source article is published open access under CC BY-NC-ND 4.0 and no part of it is reproduced in this document.

Appendix A — Core implementation

The listings below are the parts of the implementation that define the method; data generation, plotting and orchestration are omitted for space. The complete source is supplied with this report.

A.1 The three predictors

```
class AverageBaseline:
    """Market baseline: mean of the same half-hour over the previous TW days."""

    def __init__(self, tw: int):
        self.tw = tw

    def fit(self, X, y=None):
        return self

    def predict(self, X: np.ndarray) -> np.ndarray:
        return X[:, :self.tw].mean(axis=1)

class LR:
    """Linear regression, Eq. (1)."""

    def __init__(self):
        self.m = LinearRegression()

    def fit(self, X, y):
        self.m.fit(X, y)
        return self

    def predict(self, X):
        return self.m.predict(X)

class NLR:
    """Second-order polynomial (non-linear) regression, Eq. (2)."""

    def __init__(self):
        self.m = LinearRegression()

    @staticmethod
    def _phi(X: np.ndarray) -> np.ndarray:
        return np.hstack([X ** 2, X]) # squared + linear, no cross terms

    def fit(self, X, y):
        self.m.fit(self._phi(X), y)
        return self

    def predict(self, X):
        return self.m.predict(self._phi(X))
```

Listing A.1 — *models.py*. The averaging baseline is the special case of LR with all coefficients fixed at $1/TW$; NLR differs from LR only in the feature map `_phi`.

A.2 Window construction

```
def consecutive_windows(df, features, tw, target="demand", step=1):
    """Model 2: TW previous half-hours of every feature -> demand at t."""
    n = len(df) - tw - step + 1
```

```

X = np.hstack([
    np.lib.stride_tricks.sliding_window_view(df[f].to_numpy(), tw)[:n]
    for f in features])
y = df[target].to_numpy()[tw + step - 1:]
t = df.index[tw + step - 1:]
return X, y, t

def same_slot_windows(mats, target_mat, day_positions, tw, extra_per_day=None):
    """Model 1 / baseline geometry: same clock position on previous TW days."""
    rows, ys, meta = [], [], []
    for p in day_positions:
        block = np.hstack([m[p - tw:p, :].T for m in mats]) # (48, tw*len(mats))
        if extra_per_day is not None:
            block = np.hstack([block,
                               np.repeat(extra_per_day[p][None, :],
                                           SLOTS_PER_DAY, axis=0)])
        rows.append(block)
        ys.append(target_mat[p, :])
        meta.extend((p, s) for s in range(SLOTS_PER_DAY))
    return np.vstack(rows), np.concatenate(ys), meta

```

Listing A.2 — *preprocessing.py*. The forty-percent error difference reported in Section 7 is entirely a consequence of which of these two functions builds the design matrix.

A.3 Eligibility filtering and the chronological split

```

# Real DR-baseline settlement uses the last TW ELIGIBLE days: public
# holidays and event days are skipped both as targets and as window days.
festive = (((dates.month == 12) & (dates.day >= 24))
           | ((dates.month == 1) & (dates.day <= 2)))
eligible = [p for p in range(len(dates))
            if p != ev_pos and not festive[p]]

def slot_windows(targets, extra=None):
    rows, ys = [], []
    for p in targets:
        prior = [q for q in eligible if q < p][-TW:] # last TW ELIGIBLE days
        block = D[prior, :].T # (48, TW)
        if extra is not None:
            block = np.hstack(
                [block, np.repeat(extra[p][None, :], 48, axis=0)])
        rows.append(block)
        ys.append(D[p, :])
    return np.vstack(rows), np.concatenate(ys)

cand = [p for p in eligible if sum(1 for q in eligible if q < p) >= TW]
n_test = max(4, int(round(0.2 * len(cand))))
train_pos, test_pos = cand[:-n_test], cand[-n_test:] # chronological split

```

Listing A.3 — *run_dataset1.py*. The slice `[-TW:]` over the eligible list, rather than over the calendar, is the window hygiene discussed in Section 5.3.

A.4 Evaluation metric

```

def rmse(y, y_hat) -> float:
    y = np.asarray(y, dtype=float)
    y_hat = np.asarray(y_hat, dtype=float)
    return float(np.sqrt(np.mean((y - y_hat) ** 2)))

```

```
def rmse_pct(y, y_hat) -> float:  
    return 100.0 * rmse(y, y_hat) / float(np.mean(np.abs(y)))
```

Listing A.4 — evaluate.py. Equation 3, with the percentage form normalised by mean absolute actual demand so that portfolios differing in scale by two orders of magnitude can be averaged.